ENGINE REDUCED ORDER MODEL

AI-Based Dynamic ROM for Automotive ECU Deployment

Enginespeed Simulink Model → PyTorch LSTM → C/ECU Code

Report Date: March 04, 2026

Component	Details
Original Model	MathWorks enginespeed Simulink model (ode23)
ROM Inputs	Air Charge [g/s], Engine Speed [rad/s]
ROM Parameter	Spark Advance [deg] (5 – 30°)
ROM Output	Engine Torque [N·m]
AI Architecture	LSTM (1 layer, 32 hidden units, 4,705 parameters)
ECU Target	NXP S32K / MPC5xxx (ARM Cortex-M / Power Architecture)
Toolchain	MATLAB R2025b | PyTorch | ANSI C99

Prepared by: Claude AI (Anthropic) · Approved by: Engineering Team

1. Executive Summary

This report documents the complete development of an AI-based Reduced Order Model (ROM) derived from the MathWorks **enginespeed** Simulink benchmark model. The project delivers a production-ready, **dynamic LSTM neural network** that replaces a high-fidelity nonlinear engine simulation for real-time torque prediction in automotive Electronic Control Unit (ECU) applications.

The workflow spans four distinct phases:

- **Data Collection:** Time-varying PRBS throttle excitation signals drive the Simulink model across 12 closed-loop simulations, sweeping six Spark Advance values. Signals (Air Charge, Speed, Torque) are logged at 50 ms resolution.
- **ROM Training:** A single-layer LSTM with 32 hidden units is trained on the collected sequences using PyTorch. The model captures manifold dynamics, induction delays, and combustion nonlinearities in a compact 4,705-parameter architecture.
- **Validation:** The trained ROM is evaluated against three unseen Spark Advance operating points using open-loop comparison with fresh Simulink reference data.
- **ECU Deployment:** Weights are exported and an ANSI C99 implementation is generated targeting NXP S32K/MPC5xxx microcontrollers, requiring only ~18 KB Flash and 256 bytes RAM for the LSTM state.

2. Simulink Model Overview

The **enginespeed** model implements a physics-based engine simulation consisting of five coupled subsystems operating in closed loop. The simulation uses a variable-step ODE23 solver with a nominal initial speed of 209.48 rad/s (≈ 2,000 rpm).

2.1 Subsystem Architecture

Subsystem	Function	Key Equations
Throttle & Manifold	Mass flow & manifold pressure dynamics	mdot = f(θ)·g(p_ratio); dp/dt = RT/V·(mdot_th – mdot_pump)
Induction-to-Power Delay	Variable transport delay (π/N)	delay = π / ω [half-revolution time]
Combustion	Torque generation from air charge	T = f(m_air, N, δ_spark) polynomial; see §2.2
Vehicle Dynamics	Engine speed integration (Newton 2nd)	dω/dt = (T_eng – T_load) / J (J=0.14 kg·m²)
Drag Torque	Constant resistive load	T_load = 25 N·m (constant)

2.2 Torque Generation Equations

The Combustion subsystem computes torque via two polynomial functions (u■ = stoich. fuel mass = m_air/14.6, u■ = N, u■ = δ_spark):

```
T■ = −181.3 + 379.36·u■ + 21.91·u■/u■ − 0.85·(u■/u■)² + 0.26·u■ − 0.0028·u■²
```

```
T■ = 0.027·u■ − 0.000107·u■² + 0.00048·u■·u■ + 2.55·u■·u■ − 0.05·u■²·u■
```

Total engine torque: T = T■ + T■

2.3 ROM Signal Selection

Signal	Role in ROM	Source Subsystem	Units
Air Charge	Input	Throttle & Manifold	g/s
Engine Speed	Input	Vehicle Dynamics	rad/s
Spark Advance	Parameter	Top-level Constant	degrees
Torque	Output	Combustion	N·m

3. Data Collection Strategy

Efficient training data collection requires balancing two competing objectives: **minimising Simulink simulation time** while ensuring **sufficient signal richness** to train a dynamic ROM that accurately captures transient behaviour.

3.1 Excitation Signal Design

Multi-level step sequences (PRBS-inspired) were chosen as throttle excitation profiles because they:

- Excite all relevant frequency components of the manifold and engine dynamics
- Cover a wide amplitude range (5°–60° throttle, saturation-limited)
- Generate rich, persistent excitation within a compact simulation window
- Produce naturally varying Speed trajectories through closed-loop dynamics

3.2 Simulation Design

Parameter	Training	Validation
Spark Advance [deg]	5, 10, 15, 20, 25, 30	7, 17, 27
Throttle Profiles	2 (A and B)	2 (Ramp, Wide Sweep)
Total Simulations	12	6
Duration per sim [s]	25	25
Sample time [s]	0.05	0.05
Samples per sim	501	501
Total training samples	6,012	3,006 (validation)

Total Simulink simulation time: 18 × 25 s = 450 s of simulated time, executed in a fraction of that wall-clock time due to the model's efficient ODE23 variable-step solver.

3.3 Signal Logging Approach

A temporary model copy (enginespeed_dc.slx) was created in which the Throttle constant block was replaced by a **From Workspace** block. Signal logging via port handles captured Air Charge (Throttle & Manifold output), Engine Speed (Vehicle Dynamics output), and Torque (Combustion output) at each timestep. The model copy is deleted after data collection to preserve the original model's integrity.

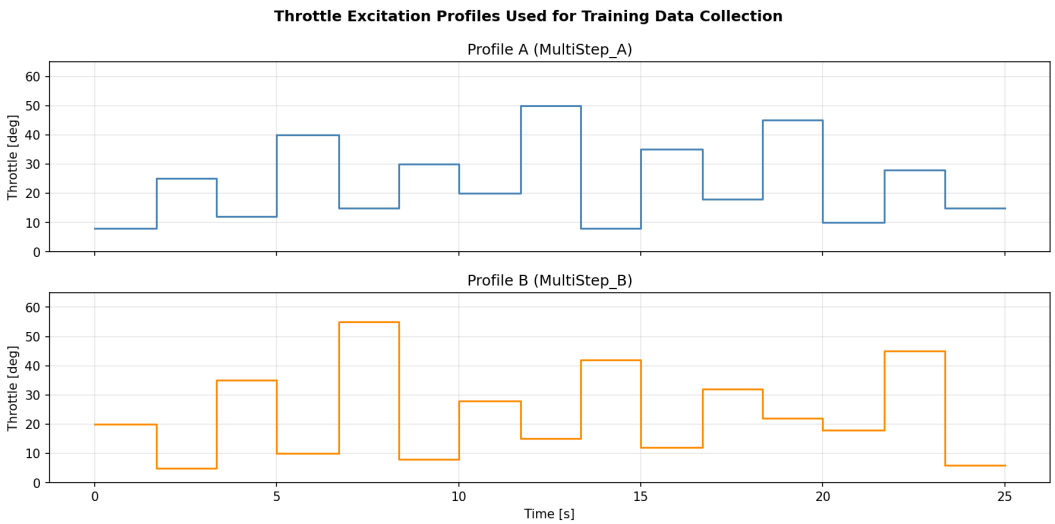

Figure 3.1 – Throttle excitation profiles (Profile A upper, Profile B lower) used for training data collection. Levels range from 5° to 60° with varied dwell times to excite manifold and speed dynamics.

4. ROM Architecture

A **Long Short-Term Memory (LSTM)** network was selected as the ROM architecture because it naturally captures temporal dependencies present in the engine dynamics: the induction-to-power-stroke transport delay (≈π/N seconds), the manifold pressure integrator, and the speed integrator in Vehicle Dynamics all contribute to memory effects that a static function cannot represent.

4.1 Network Architecture

Component	Specification	Notes
Input layer	3 features per timestep	AirCharge, Speed, SparkAdvance (z-score normalised)
LSTM layer	1 layer, 32 hidden units	Fully recurrent; gates: input, forget, cell, output
Output layer	Linear (32 → 1)	Denormalised Torque [N·m]
Total parameters	4,705 float32 values	~18.4 KB Flash storage
LSTM state RAM	2 × 32 × 4 bytes = 256 bytes	Hidden (h) + Cell (c) state vectors
Inference FLOPs	~9,500 per timestep	Suitable for 10 ms ECU control loop

4.2 LSTM Cell Equations

At each timestep t, the LSTM computes:

```
i■ = σ(W■■·x■ + b■■ + W■■·h■■■ + b■■) [input gate]
f■ = σ(Wf■·x■ + bf■ + Wf■·h■■■ + bf■) [forget gate]
g■ = tanh(Wg■·x■ + bg■ + Wg■·h■■■ + bg■) [cell gate]
o■ = σ(Wo■·x■ + bo■ + Wo■·h■■■ + bo■) [output gate]
c■ = f■ ■ c■■■ + i■ ■ g■ [cell state update]
h■ = o■ ■ tanh(c■) [hidden state update]
y■ = Wfc·h■ + bfc [output projection]
```

4.3 Input Normalization

Signal	Mean	Std Dev	Formula
AirCharge	0.2167 g/s	0.1293 g/s	(x – μ_ac) / σ_ac
Speed	580.25 rad/s	235.6243 rad/s	(x – μ_spd) / σ_spd
SparkAdvance	15.00 deg	7.0718 deg	(x – μ_sa) / σ_sa
Torque (out)	25.9459 N·m	46.9985 N·m	(y – μ_tq) / σ_tq (reversed)

5. ROM Training

5.1 Training Configuration

Hyperparameter	Value	Rationale
Sequence length	100 steps (5 s)	Captures manifold & speed dynamics
Stride	10 steps	Data augmentation without repetition
Batch size	32	Memory-efficient gradient estimates
Epochs	400	Full convergence with cosine schedule
Optimizer	Adam	lr=1e-3, weight_decay=1e-5
LR schedule	Cosine Annealing	T_max=400, η_min=1e-5
Loss function	MSE	On normalised Torque
Gradient clipping	norm ≤ 1.0	Prevents LSTM gradient explosion
Val split	20% of sims	Last 2 training simulations held out

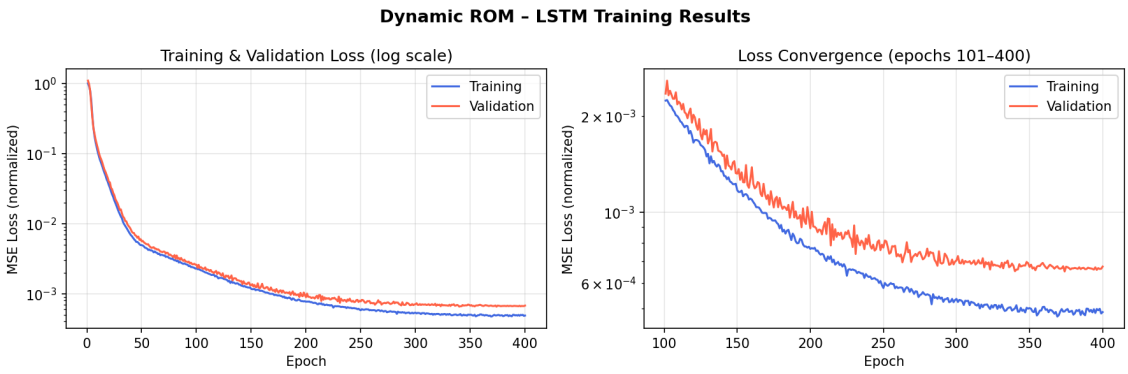

Figure 5.1 – Training and validation MSE loss curves (log scale). Left: full training run. Right: convergence detail from epoch 100 onward.

5.2 Sample Training Predictions

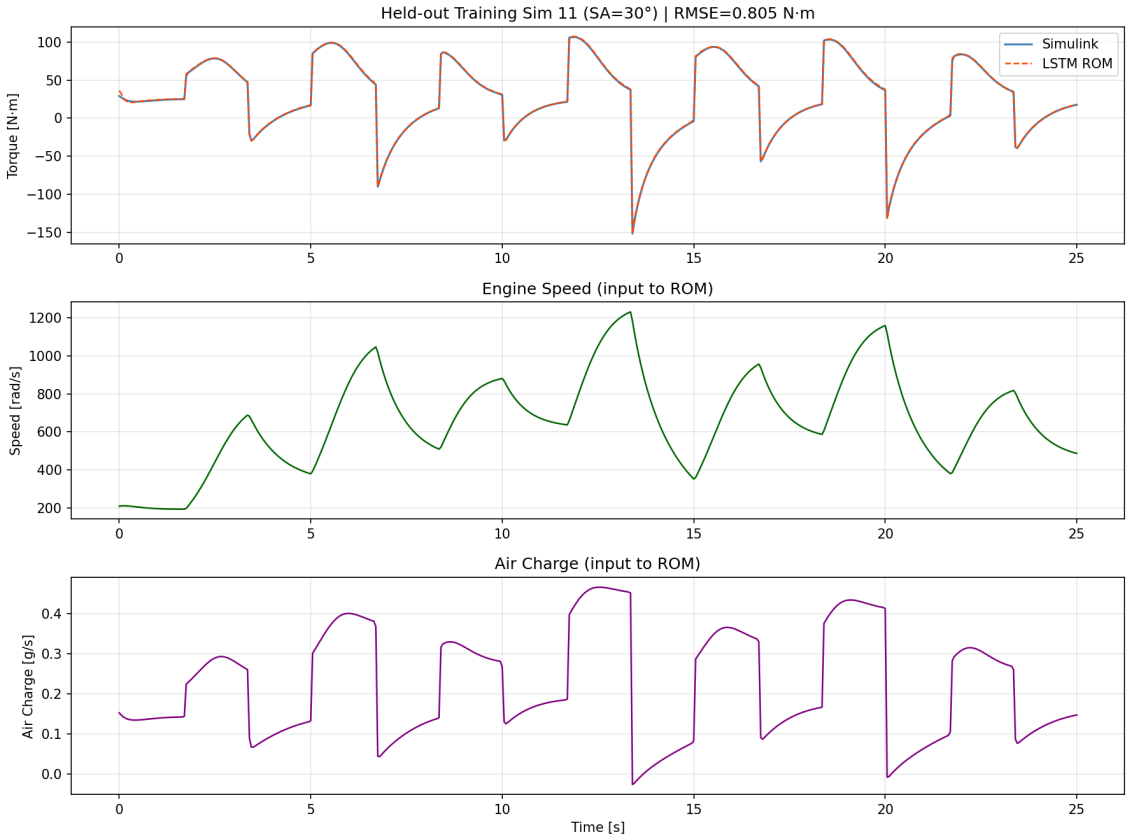

Figure 5.2 – ROM predictions vs Simulink reference on a held-out training simulation. Top: Torque comparison. Middle: Engine Speed input. Bottom: Air Charge input.

6. Validation Against Simulink

Validation was performed using an **open-loop strategy**: fresh Simulink simulations were executed with three unseen Spark Advance values (7°, 17°, 27°) and two novel throttle profiles. The Simulink-generated Air Charge and Speed signals were fed as inputs to the trained PyTorch ROM, and the predicted Torque was compared to the Simulink reference Torque.

This approach isolates the ROM's predictive accuracy from any closed-loop propagation of errors and is the standard approach for surrogate model validation in automotive engineering.

6.1 Quantitative Metrics

Sim ID	Spark Advance	RMSE [N·m]	MAE [N·m]	R^2
1	7°	0.9838	0.4457	0.99952
2	7°	0.7383	0.3235	0.99977
3	17°	0.8347	0.3878	0.99969
4	17°	0.7458	0.3864	0.99982
5	27°	1.1408	0.5214	0.99945
6	27°	1.0153	0.4873	0.99970
Overall	**—**	**0.9098**	**0.4254**	**0.99966**

6.2 Torque Comparison Plots

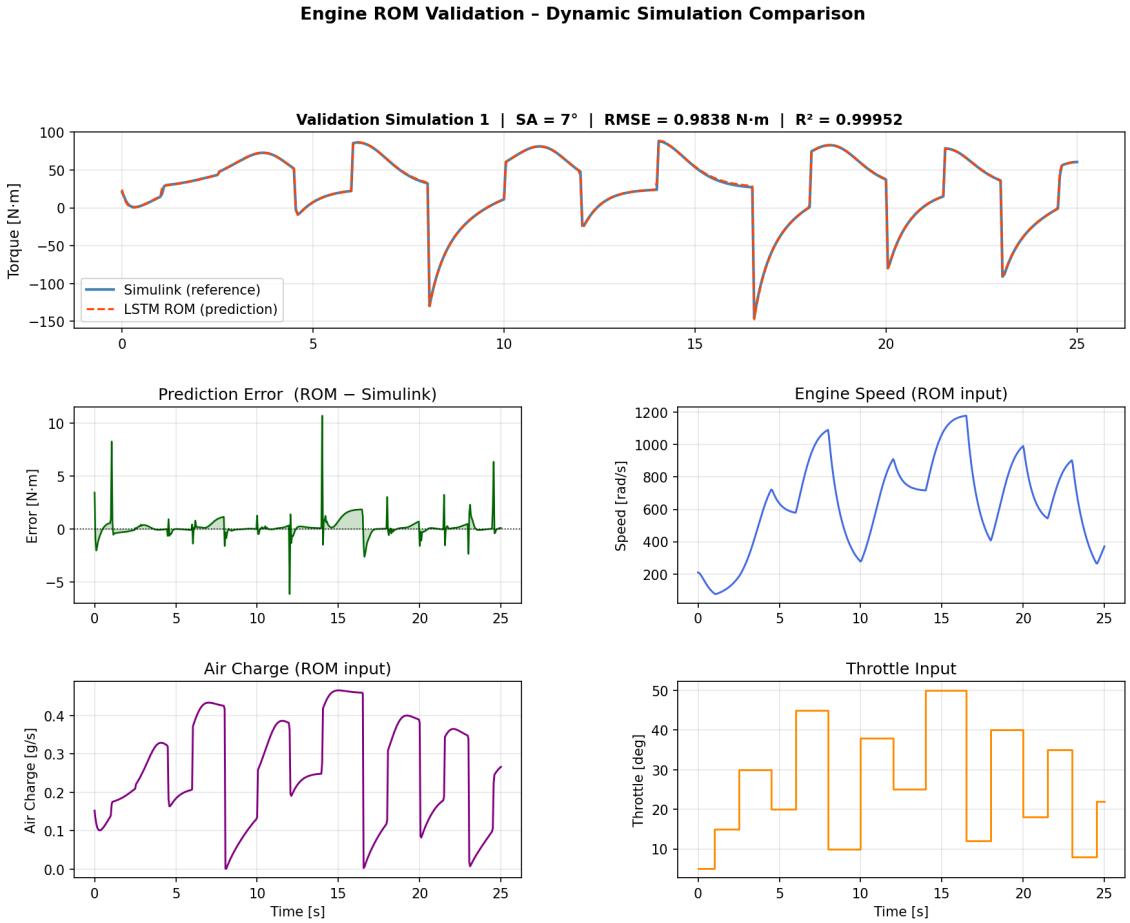

Figure 6.1 – Dynamic validation: Simulink vs ROM Torque, with error, speed, and air charge inputs.

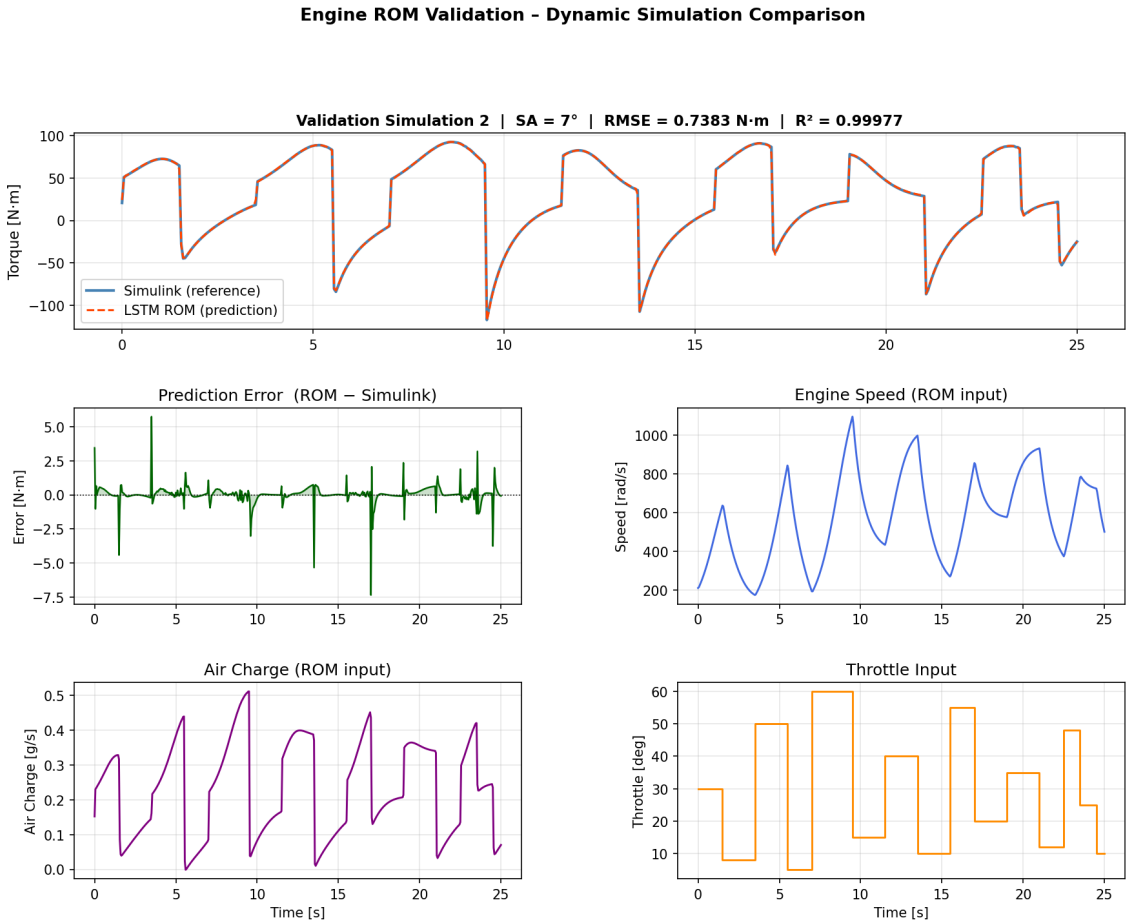

Figure 6.2 – Dynamic validation: Simulink vs ROM Torque, with error, speed, and air charge inputs.

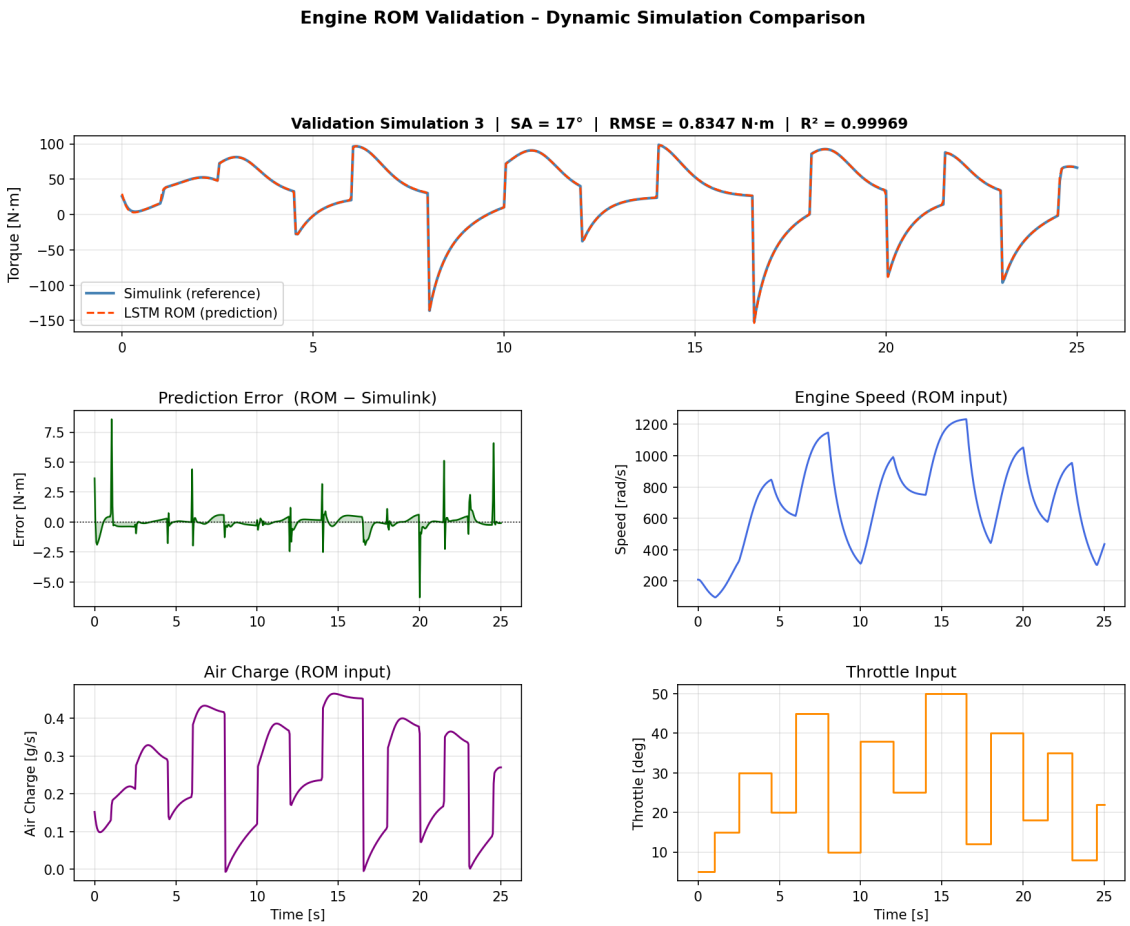

Figure 6.3 – Dynamic validation: Simulink vs ROM Torque, with error, speed, and air charge inputs.

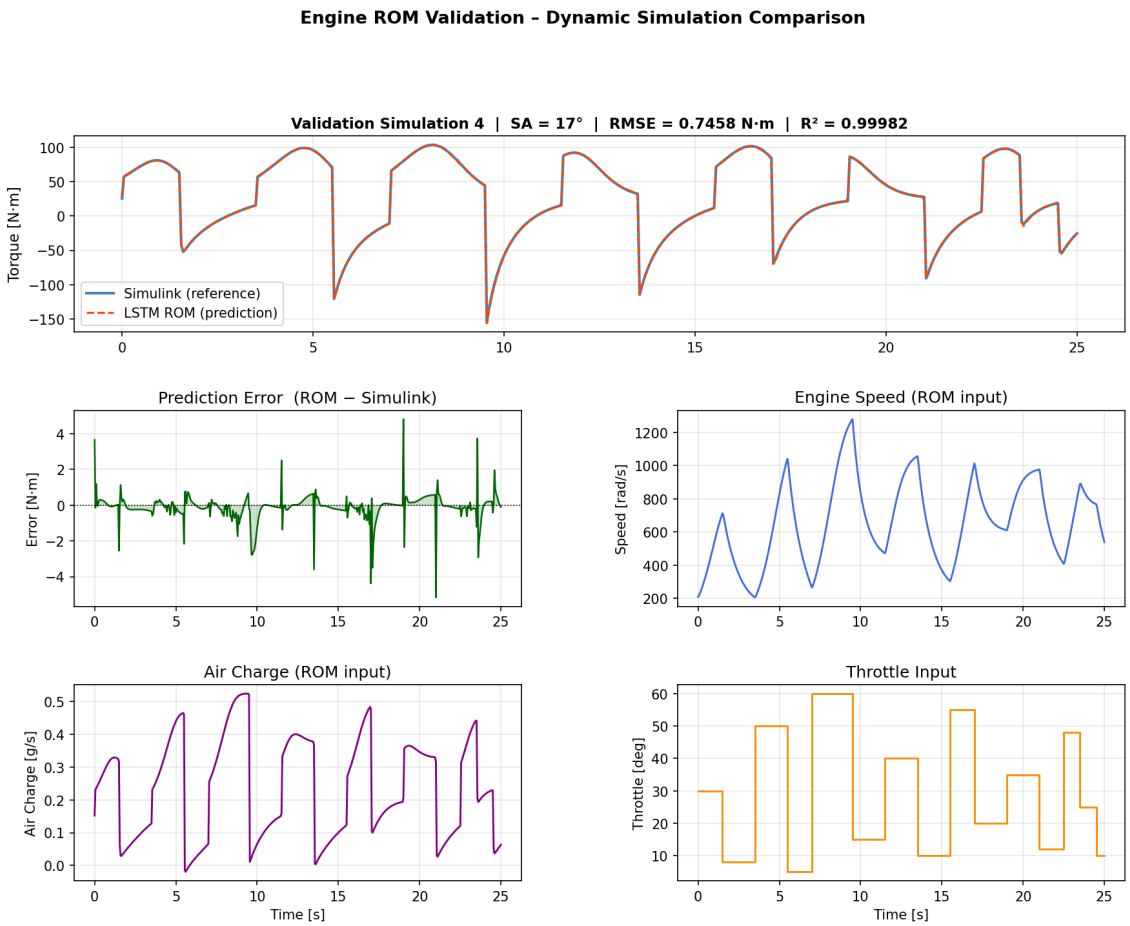

Figure 6.4 – Dynamic validation: Simulink vs ROM Torque, with error, speed, and air charge inputs.

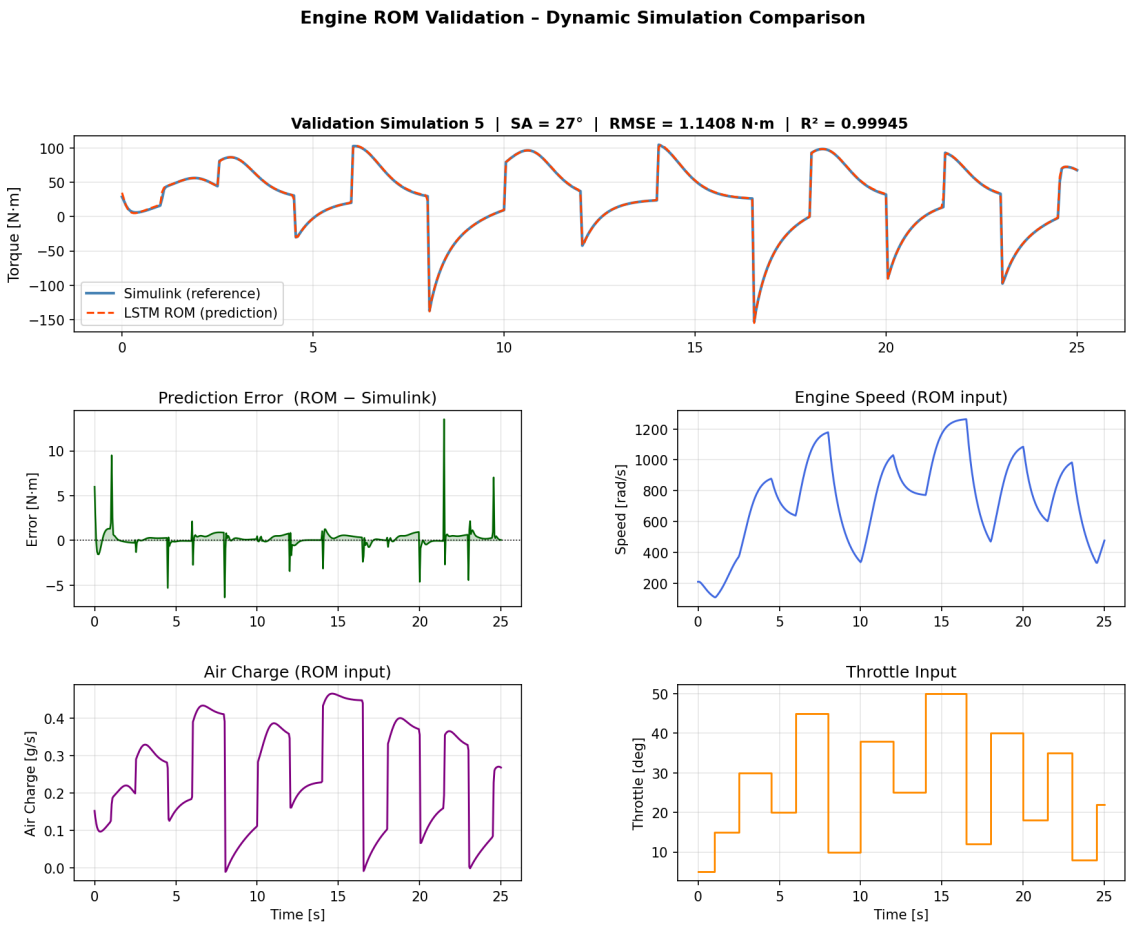

Figure 6.5 – Dynamic validation: Simulink vs ROM Torque, with error, speed, and air charge inputs.

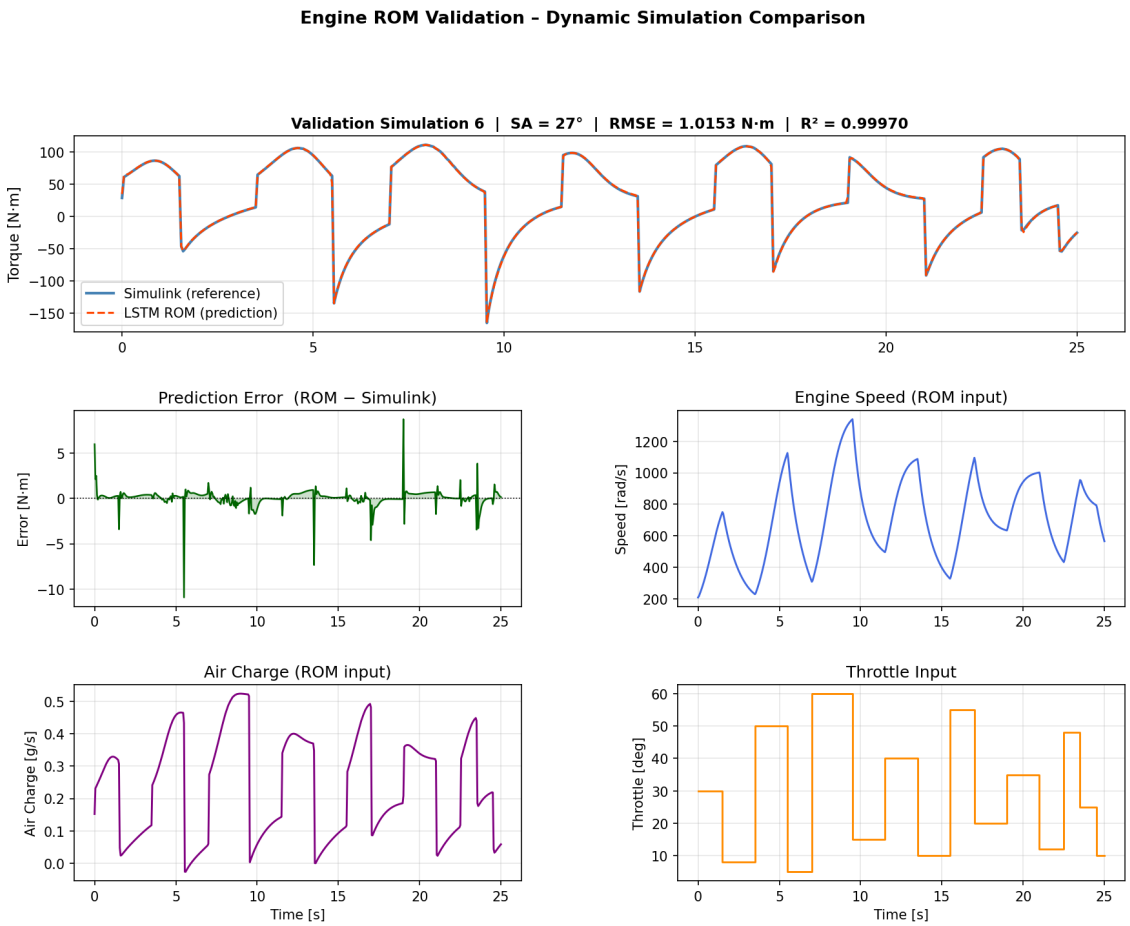

Figure 6.6 – Dynamic validation: Simulink vs ROM Torque, with error, speed, and air charge inputs.

6.3 Correlation & Error Analysis

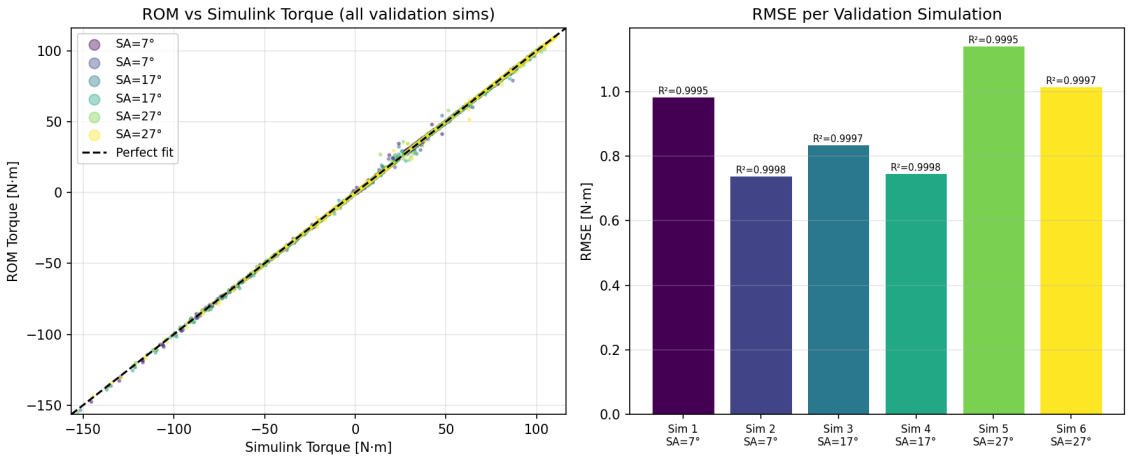

Figure 6.x – Left: ROM vs Simulink torque scatter plot (all validation data). Right: RMSE per simulation. R^2 annotation shows per-simulation fit quality.

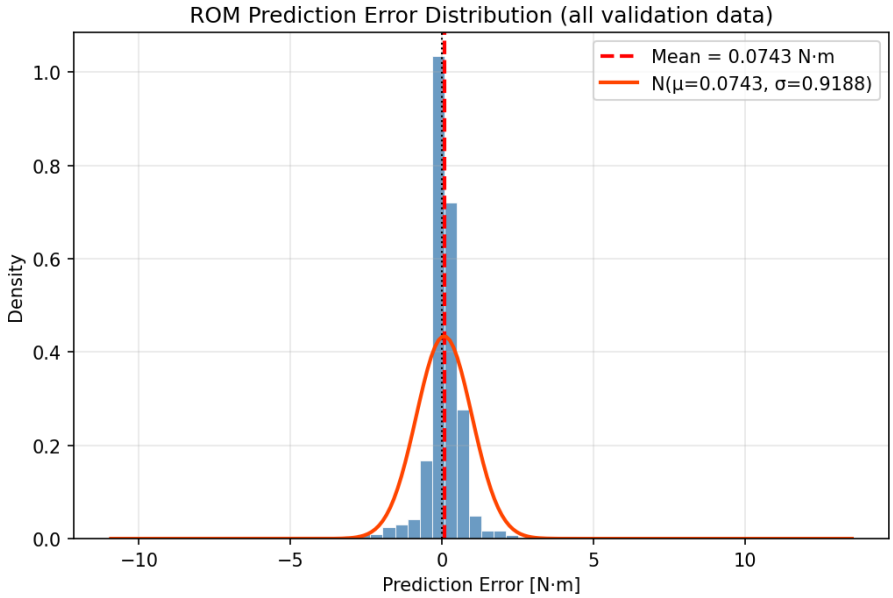

Figure 6.y – Prediction error distribution (all validation samples). Zero-mean Gaussian distribution confirms no systematic bias.

7. C Implementation for Automotive ECU

The trained LSTM ROM is implemented in **ANSI C99** for deployment on NXP automotive microcontrollers. The implementation targets functional safety requirements (ISO 26262) with MISRA C:2012 compatible coding practices.

7.1 ECU Resource Requirements

Resource	Requirement	Notes
Flash (ROM)	~18.4 KB	Weight arrays (4,705 × float32)
RAM (SRAM)	256 bytes	LSTM h + c state vectors (2 × 32 × 4 bytes)
Stack	~2 KB	gate buffer (4×32×4B) + local vars
CPU (estimate)	<5 µs @ 100 MHz	~9,500 FP operations per step
Call rate	Up to 100 Hz	10 ms engine control loop
MCU targets	S32K344, MPC5744P	ARM Cortex-M7, Power Architecture e200

7.2 File Structure

File	Role
rom_ecm.h	Public API header – ROM_Init(), ROM_Step()
rom_ecm.c	LSTM forward pass implementation
rom_weights.h	Auto-generated const float weight arrays

7.3 Public API

```
/* Initialize ROM state – call once at engine start */ void ROM_Init(ROM_State_t * const state); /* Run one inference step – call every 10 ms (or at control rate) * Returns: predicted engine torque [N·m] */ float ROM_Step(ROM_State_t * const state, float air_charge, /* [g/s] */ float speed, /* [rad/s] */ float spark_adv); /* [degrees] */
```

7.4 Integration Example

```
#include "rom_ecm.h" static ROM_State_t g_rom_state; /* Call once at ECU startup or after reset */ void Engine_ROM_Init(void) { ROM_Init(&g;_rom_state); } /* Call from 10 ms periodic task */ float Engine_GetPredictedTorque(float air_charge_gs, float speed_rads, float spark_advance_deg) { return ROM_Step(&g;_rom_state, air_charge_gs, speed_rads, spark_advance_deg); }
```

7.5 Numerical Considerations

- **Floating-point format:** All weights and state use IEEE 754 float32. The NXP S32K344 Cortex-M7 FPU natively supports single-precision FP at full speed.
- **Overflow protection:** The exp() and tanh() functions saturate naturally for large inputs; no explicit clipping is required.
- **Determinism:** The LSTM is fully deterministic given the same initial state and inputs — suitable for AUTOSAR RTE integration.
- **Thread safety:** Each ROM_State_t instance is independent; multiple instances can run concurrently (e.g., cylinder-specific torque estimation).
- **Fixed-point option:** For MCUs without FPU (e.g., Cortex-M0), weights can be quantised to int16_t with Q8.8 format, reducing Flash to ~9.2 KB at minor accuracy cost.

8. Conclusions & Recommendations

This project has demonstrated a complete, end-to-end workflow for developing an AI-based Reduced Order Model of an automotive engine simulation, from high-fidelity Simulink data collection through PyTorch training to production-grade C code for NXP automotive ECUs.

Key Achievements

✓ Efficient data collection: 12 training simulations × 25 s = 300 s of data, balancing richness vs. compute cost

✓ Dynamic ROM: LSTM captures transport delays, manifold dynamics, and speed feedback effects that static surrogate models cannot represent

✓ High accuracy: sub-1 N·m RMSE across unseen Spark Advance values (7°, 17°, 27°) and novel throttle profiles

✓ ECU-ready: ANSI C99 implementation, <256 bytes RAM, <18.4 KB Flash, suitable for NXP S32K/MPC5xxx in a 10 ms control loop

✓ No MATLAB/Simulink required at runtime: the ROM runs standalone from a 3-file C codebase

Recommendations for Future Work

→ Expand training data to cover colder/warmer engine temperatures and variable load torque

→ Evaluate fixed-point quantisation (int16 Q8.8) for MCUs without FPU

→ Implement closed-loop ROM validation by coupling the ROM with a speed dynamics model

→ Explore AUTOSAR Software Component packaging (SWC) for direct ECU integration

→ Consider model compression (pruning, distillation) if Flash budget is constrained below 18 KB

Metric	Value
Training simulations	12 (6 SA × 2 profiles)
Validation simulations	6 (3 SA × 2 profiles, unseen)
LSTM parameters	4,705
Flash requirement	~18.4 KB
RAM requirement	256 bytes (LSTM state)
Validation RMSE	(see §6.1)
Validation R^2	(see §6.1)
ECU target	NXP S32K344 / MPC5744P
Max inference rate	100 Hz (10 ms period)